

BiteFixes Enterprise Architecture 2.0

La filosofía

BiteFixes no es un sistema.

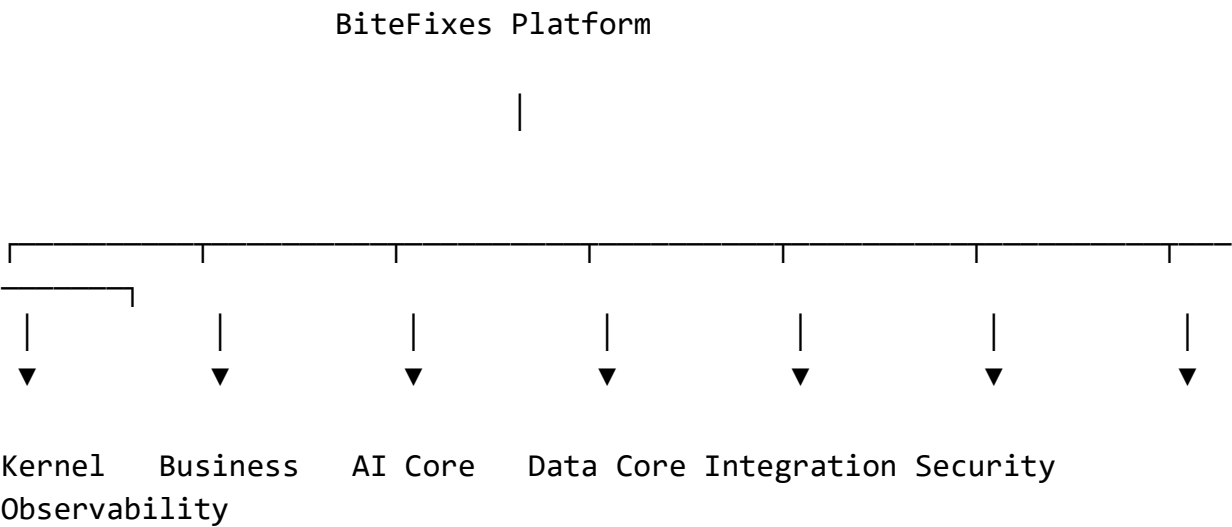
Tampoco es una API.

Tampoco es un chatbot.

Es una plataforma empresarial basada en dominios.

Los 7 Núcleos de BiteFixes

En lugar de dividir únicamente por carpetas, propongo dividir la plataforma en siete núcleos claramente definidos.



Cada núcleo tiene una responsabilidad exclusiva.

1. Kernel

Es el corazón del sistema.

Nunca contiene lógica de negocio.

Responsabilidades:

- Ciclo de vida de la aplicación.
- Gestión de módulos.
- Configuración.
- Eventos.
- Inyección de dependencias.
- Contexto de empresa.
- Contexto de usuario.
- Auditoría.

Nunca conoce qué es un Ticket.

2. Business Core

Aquí vive el negocio.

Customer

Device

Service

Ticket

Inventory

Knowledge

Billing

CRM

Reports

Appointments

Warranty

Cada dominio posee:

Entity

Repository

Policies

Use Cases

Events

Validators

DTO

Services

3. AI Core

Aquí vive Bitey.

No contiene reglas del negocio.

Solo inteligencia.

Language

Intent

Memory

Planning

Decision

Reasoning

Knowledge Retrieval

Recommendation

Learning

Reflection

El AI Core nunca modifica directamente una tabla.

Siempre solicita acciones al Business Core.

4. Data Core

Uno de los mayores cambios que introduciría.

No tendría un único acceso a PostgreSQL.

Tendría varios almacenes especializados.

PostgreSQL

↓

Business Data

Redis



Cache



Object Storage



Images

Files



Vector Store



Embeddings



Analytics Store



KPIs

Metrics

Cada tipo de dato vive donde corresponde.

5. Integration Core

Toda comunicación externa.

WhatsApp

Instagram

Facebook

Telegram

Email

SMS

REST

Webhooks

OpenAI

Gemini

Stripe

Mercado Pago

Nunca mezclada con la lógica del negocio.

6. Security Core

No solamente autenticación.

También:

Authentication

Authorization

Permissions

Roles

Policies

Encryption

Secrets

Audit

Compliance

Preparado para crecer hacia requisitos empresariales.

7. Observability Core

Aquí vive todo lo relacionado con el funcionamiento interno.

Logs

Metrics

Tracing

Performance

Alerts

Health Checks

Business KPIs

Permite saber qué ocurre en la plataforma en tiempo real.

Arquitectura por Dominios

Cada dominio mantiene exactamente la misma estructura.

Ejemplo:

`ticket/`

`entities/`

`repositories/`

`use_cases/`

`events/`

`policies/`

`validators/`

`dto/`

`api/`

`tests/`

Si mañana se añade `inventory/`, seguirá el mismo patrón.

Unificación de eventos

En lugar de eventos aislados, definiría un catálogo oficial.

Por ejemplo:

CustomerRegistered

CustomerUpdated

ConversationStarted

ConversationClosed

MessageReceived

MessageSent

IntentDetected

KnowledgeMatched

TicketOpened

TicketAssigned

TicketClosed

EstimateGenerated

PaymentReceived

WarrantyActivated

Todos con un formato común.

Modelo de permisos

En lugar de roles rígidos, usaría permisos granulares.

Permission

↓

ticket.read

ticket.create

ticket.close

customer.update

knowledge.edit

inventory.manage

Los roles agrupan permisos.

Technician

↓

ticket.read

ticket.update

device.read

Manager

↓

Todo lo anterior

+

`ticket.assign`

`reports.view`

Esto hace el sistema mucho más flexible.

Motor de automatización

No escribir automatizaciones en código.

Diseñar un motor configurable.

Ejemplo:

Evento

↓

Ticket Closed

↓

Condición

↓

Warranty = true

↓

Acción

↓

Enviar correo

↓

Esperar 7 días

↓

Enviar encuesta

Cada empresa puede adaptar sus procesos sin modificar el backend.

Motor de reglas

Separaría las reglas del código.

Ejemplo:

IF

Customer is VIP

AND

Repair Cost > 500

THEN

Require Manager Approval

El backend interpreta estas reglas; no es necesario recompilar para cambiarlas.

Centro de conocimiento

La base de conocimiento dejaría de ser solo preguntas y respuestas.

La organizaría como un grafo de información.

Notebook



Symptoms



Possible Causes



Diagnostic Tests



Recommended Repairs



Required Materials



Estimated Time



Warranty

Esto mejora las recomendaciones y facilita futuras búsquedas semánticas.

Mi recomendación para la siguiente etapa

Hasta ahora hemos trabajado principalmente desde la perspectiva del software. El siguiente paso sería empezar a modelar **la empresa que usará el software**.

Eso implica definir aspectos como:

- Procesos de recepción.
- Diagnóstico.
- Reparación.
- Gestión de garantías.
- Atención al cliente.
- Agenda de técnicos.
- Compras y stock.
- Facturación.
- Indicadores de desempeño.

La plataforma no solo debe administrar datos; debe representar de forma fiel cómo funciona un negocio de asistencia técnica. Si el modelo empresarial es sólido, los módulos, las APIs y Bitye podrán evolucionar sobre una base consistente durante muchos años. ²memcite